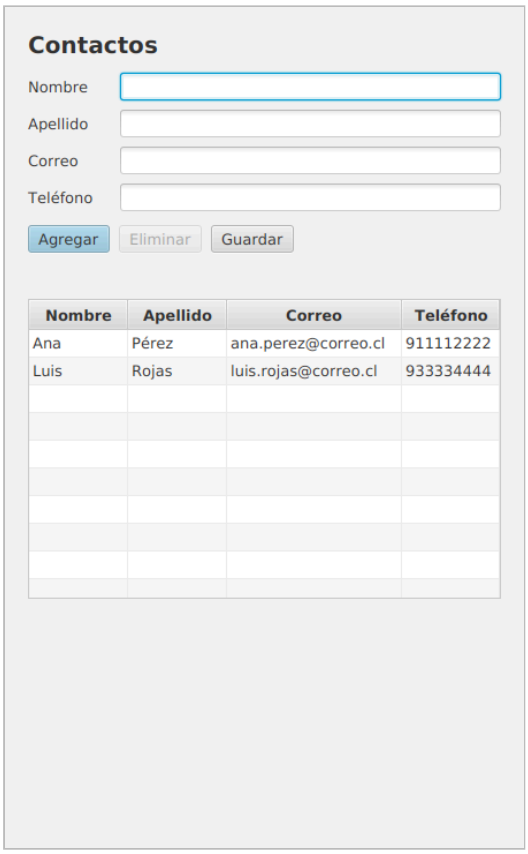
JavaFX: formulario + TableView + JSON

Proyecto **3_javafx-table** · JavaFX 21 · JDK 21 · Maven · Gson · Programación Orientada a Objetos (DUOC UC)

0. Qué vamos a construir

Una agenda de **contactos**. A partir del proyecto base (con el `pom.xml`, los controllers vacíos y un FXML incompleto) vas a agregar:

- Un **formulario** con 4 campos: nombre, apellido, correo y teléfono.
- Un **TableView** conectado a una lista *observable*: cuando agregas o eliminas un contacto, la tabla se actualiza sola.
- Un botón **Eliminar** que borra la fila seleccionada.
- Un botón **Guardar** que escribe los contactos en `datos/contactos.json` (con **Gson**). Al abrir la aplicación de nuevo, los contactos se cargan desde ese archivo.



Resultado esperado: la aplicación recién abierta, con los contactos cargados desde el JSON.

Estructura final del proyecto

```
3_javafx-table/
├── pom.xml                                    (modificas: agregar Gson)
├── datos/
│   └── contactos.json                         (se crea solo al presionar Guardar)
└── src/main/
    ├── java/
    │   ├── module-info.java                   (modificas)
    │   └── com/duoc/javafxtable/
    │       ├── TablaApplication.java          (ya viene hecha)
    │       ├── ContactoController.java        (vacío: lo completas)
    │       ├── Contacto.java                  (nueva)
    │       └── ContactoRepositorio.java       (nueva)
    └── resources/com/duoc/javafxtable/
        └── contacto-view.fxml                 (incompleto: lo completas)
```

1. Abrir y ejecutar el proyecto base

1. Abre la carpeta `3_javafx-table` en IntelliJ (*File* → *Open*, selecciona el `pom.xml` y elige *Open as Project*).
2. Espera a que Maven descargue las dependencias.
3. Ejecuta `TablaApplication` (o en la terminal: `mvn javafx:run`).

Debe abrir una ventana que solo muestra el título **Contactos**. Ese es el punto de partida.

2. Paso 1: agregar Gson al pom.xml

Gson es una librería de Google que convierte objetos Java a JSON y al revés. Como no viene con Java, hay que declararla como dependencia. En `pom.xml`, reemplaza el comentario `PASO 1` (dentro de `<dependencies>`) por:

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
</dependency>
```

Luego recarga Maven (en IntelliJ: el botón *Load Maven Changes*, o `Ctrl+Shift+O`).

3. Paso 2: module-info.java

El proyecto es modular, así que cada librería que usamos hay que declararla, y cada paquete al que una librería accede por reflexión hay que "abrirlo". Esta es la parte donde más se falla.

Quién	Para qué necesita acceso
`javafx.fxml`	Crear el controller e inyectar los campos `@FXML`.

`javafx.base`	`PropertyValueFactory` busca el getter `getNombre()` de `Contacto`.
`com.google.gson`	Leer y escribir los campos de `Contacto` al convertir a JSON.

`src/main/java/module-info.java`

```
module com.duoc.javafxtable {
    requires javafx.controls;
    requires javafx.fxml;
    requires com.google.gson;
    requires java.desktop;

    // javafx.fxml: crea el controlador e inyecta los @FXML
    // javafx.base: la tabla lee los getters de Contacto (PropertyValueFactory)
    // com.google.gson: lee y escribe los campos de Contacto
    opens com.duoc.javafxtable to javafx.fxml, javafx.base, com.google.gson;
    exports com.duoc.javafxtable;
}
```

> **Error típico:** si olvidas el `opens ... to com.google.gson`, al guardar aparece `InaccessibleObjectException`. Si olvidas `javafx.base`, la tabla se ve vacía aunque tenga datos.

4. Paso 3: la clase Contacto (modelo)

Crea la clase `Contacto` en el paquete `com.duoc.javafxtable`. Son 4 campos privados, un constructor y los getters. Los getters son los que usa la tabla: la columna "nombre" llama a `getNombre()`.

`src/main/java/com/duoc/javafxtable/Contacto.java`

```
package com.duoc.javafxtable;

public class Contacto {

    private String nombre;
    private String apellido;
    private String correo;
    private String telefono;

    public Contacto(String nombre, String apellido, String correo, String telefono) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.correo = correo;
        this.telefono = telefono;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public String getCorreo() {
        return correo;
    }

    public String getTelefono() {
        return telefono;
    }
}
```

5. Paso 4: ContactoRepositorio (guardar y cargar JSON)

Esta clase se encarga solo del archivo. El controller no sabe nada de Gson: solo pide `cargar()` y `guardar(lista)`.

`src/main/java/com/duoc/javafxtable/ContactoRepositorio.java`

```
package com.duoc.javafxtable;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;

import java.io.IOException;
import java.lang.reflect.Type;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

/** Lee y escribe la lista de contactos en datos/contactos.json. */
public class ContactoRepositorio {

    private static final Path ARCHIVO = Path.of("datos", "contactos.json");
    private static final Type TIPO_LISTA = new TypeToken<List<Contacto>>() { }.getType();
```

```
    private final Gson gson = new GsonBuilder().setPrettyPrinting().create();

    public List<Contacto> cargar() {
        if (!Files.exists(ARCHIVO)) {
            return new ArrayList<>();
        }
        try {
            String json = Files.readString(ARCHIVO);
            List<Contacto> contactos = gson.fromJson(json, TIPO_LISTA);
            return contactos != null ? contactos : new ArrayList<>();
        } catch (IOException e) {
            throw new RuntimeException("No se pudo leer " + ARCHIVO, e);
        }
    }

    public void guardar(List<Contacto> contactos) throws IOException {
        Files.createDirectories(ARCHIVO.getParent());
        // new ArrayList: Gson recibe una lista normal en vez de la ObservableList
        Files.writeString(ARCHIVO, gson.toJson(new ArrayList<>(contactos)));
    }
}
```

- `TypeToken` le dice a Gson que el JSON es una **lista de Contacto** (por el borrado de tipos de Java, `List<Contacto>.class` no existe).
- `setPrettyPrinting()` deja el JSON con sangría para que sea legible.
- Si el archivo no existe (primera ejecución), `cargar()` devuelve una lista vacía.

6. Paso 5: completar contacto-view.fxml

Abre `contacto-view.fxml` en Scene Builder (clic derecho → *Open in SceneBuilder*) o edítalo a mano. Dentro del `VBox`, bajo el título, agrega **en este orden**:

1. **GridPane** con 4 filas: un `Label` (col. 0) y un `TextField` (col. 1) por fila. fx:id: `nombreField`, `apellidoField`, `correoField`, `telefonoField`.
2. **HBox** con 3 `Button`: *Agregar* (`onAction="#onAgregarClick"`), *Eliminar* (`fx:id="eliminarButton"`, `onAction="#onEliminarClick"`) y *Guardar* (`onAction="#onGuardarClick"`).
3. **Label** `fx:id="mensajeLabel"` para errores y avisos.
4. **TableView** `fx:id="tabla"` con 4 `TableColumn`: `nombreCol`, `apellidoCol`, `correoCol`, `telefonoCol`.

Resultado completo del FXML:

`src/main/resources/com/duoc/javafxtable/contacto-view.fxml`

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.control.TextField?>
```

```
<?import javafx.scene.layout.ColumnConstraints?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>

<VBox spacing="12.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.duoc.javafxtable.ContactoController">
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>
   <children>
      <Label text="Contactos" style="-fx-font-size: 20px; -fx-font-weight: bold;" />

      <GridPane hgap="10.0" vgap="8.0">
         <columnConstraints>
            <ColumnConstraints minWidth="70.0" />
            <ColumnConstraints hgrow="ALWAYS" />
         </columnConstraints>
         <children>
            <Label text="Nombre" GridPane.rowIndex="0" />
            <TextField fx:id="nombreField" GridPane.columnIndex="1" GridPane.rowIndex="0"
/>
            <Label text="Apellido" GridPane.rowIndex="1" />
            <TextField fx:id="apellidoField" GridPane.columnIndex="1" GridPane.rowIndex="1"
/>
            <Label text="Correo" GridPane.rowIndex="2" />
            <TextField fx:id="correoField" GridPane.columnIndex="1" GridPane.rowIndex="2"
/>
            <Label text="Teléfono" GridPane.rowIndex="3" />
            <TextField fx:id="telefonoField" GridPane.columnIndex="1" GridPane.rowIndex="3"
/>
         </children>
      </GridPane>

      <HBox spacing="8.0">
         <children>
            <Button defaultButton="true" onAction="#onAgregarClick" text="Agregar" />
            <Button fx:id="eliminarButton" onAction="#onEliminarClick" text="Eliminar" />
            <Button onAction="#onGuardarClick" text="Guardar" />
         </children>
      </HBox>

      <Label fx:id="mensajeLabel" />

      <TableView fx:id="tabla" prefHeight="260.0">
         <columns>
            <TableColumn fx:id="nombreCol" prefWidth="120.0" text="Nombre" />
            <TableColumn fx:id="apellidoCol" prefWidth="120.0" text="Apellido" />
            <TableColumn fx:id="correoCol" prefWidth="220.0" text="Correo" />
            <TableColumn fx:id="telefonoCol" prefWidth="120.0" text="Teléfono" />
         </columns>
         <columnResizePolicy>
            <TableView fx:constant="CONSTRAINED_RESIZE_POLICY" />
         </columnResizePolicy>
      </TableView>
   </children>
</VBox>
```

> Si usas Scene Builder, recuerda que cada `fx:id` y cada método de `onAction` deben escribirse **exactamente igual** que en el controller (mayúsculas incluidas), y que los `import` del FXML los agrega Scene Builder solo.

7. Paso 6: campos @FXML del controller

Abre `ContactoController`. Por cada `fx:id` del FXML, declara un campo con el mismo nombre y la anotación `@FXML`. Aquí también va la lista observable y el repositorio:

```
@FXML private TextField nombreField;
@FXML private TextField apellidoField;
@FXML private TextField correoField;
@FXML private TextField telefonoField;
@FXML private Button eliminarButton;
@FXML private Label mensajeLabel;

@FXML private TableView<Contacto> tabla;
@FXML private TableColumn<Contacto, String> nombreCol;
@FXML private TableColumn<Contacto, String> apellidoCol;
@FXML private TableColumn<Contacto, String> correoCol;
@FXML private TableColumn<Contacto, String> telefonoCol;

// Lista "observable": la tabla se redibuja sola cuando cambia
private final ObservableList<Contacto> contactos = FXCollections.observableArrayList();
private final ContactoRepositorio repositorio = new ContactoRepositorio();
```

8. Paso 7: initialize() — conectar tabla y lista

El método `initialize()` lo llama JavaFX automáticamente una vez cargado el FXML. Aquí hacemos tres cosas:

```
@FXML
private void initialize() {
    // 1. Cada columna sabe qué getter leer ("nombre" -> getNombre())
    nombreCol.setCellValueFactory(new PropertyValueFactory<>("nombre"));
    apellidoCol.setCellValueFactory(new PropertyValueFactory<>("apellido"));
    correoCol.setCellValueFactory(new PropertyValueFactory<>("correo"));
    telefonoCol.setCellValueFactory(new PropertyValueFactory<>("telefono"));

    // 2. Cargamos el JSON a la lista y se la entregamos a la tabla
    contactos.addAll(repositorio.cargar());
    tabla.setItems(contactos);

    // 3. Eliminar queda deshabilitado mientras no haya una fila seleccionada
    eliminarButton.disableProperty()
            .bind(tabla.getSelectionModel().selectedItemProperty().isNull());
}
```

> **La clave de "observable":** `tabla.setItems(contactos)` une la tabla con la lista. Después, solo haces `contactos.add(...)` o `contactos.remove(...)` y la tabla cambia sola; nunca la actualizas a mano.

9. Paso 8: Agregar (con validaciones) y Eliminar

```
@FXML
private void onAgregarClick() {
    String nombre = nombreField.getText().trim();
    String apellido = apellidoField.getText().trim();
    String correo = correoField.getText().trim();
    String telefono = telefonoField.getText().trim();

    if (nombre.isEmpty() || apellido.isEmpty() || correo.isEmpty() || telefono.isEmpty()) {
        mostrarError("Debes completar todos los campos.");
        return;
    }
    if (!correo.contains("@") || !correo.contains(".")) {
        mostrarError("El correo no es válido.");
        return;
    }
    if (!telefono.matches("\\d{8,12}")) {
        mostrarError("El teléfono debe tener entre 8 y 12 dígitos.");
        return;
    }

    contactos.add(new Contacto(nombre, apellido, correo, telefono));
    limpiarFormulario();
    mostrarAviso("Contacto agregado. Presiona Guardar para escribirlo en el archivo.");
}

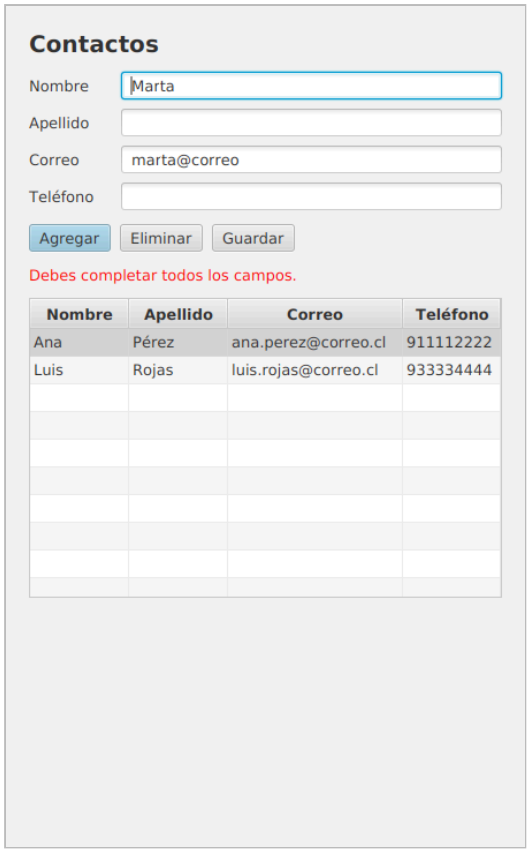
@FXML
private void onEliminarClick() {
    Contacto seleccionado = tabla.getSelectionModel().getSelectedItem();
    if (seleccionado != null) {
        contactos.remove(seleccionado);
        mostrarAviso("Contacto eliminado. Presiona Guardar para escribirlo en el
archivo.");
    }
}
```

Y los métodos auxiliares (al final de la clase):

```
private void limpiarFormulario() {
    nombreField.clear();
    apellidoField.clear();
    correoField.clear();
    telefonoField.clear();
    nombreField.requestFocus();
}

private void mostrarError(String texto) {
    mensajeLabel.setStyle("-fx-text-fill: red;");
    mensajeLabel.setText(texto);
}

private void mostrarAviso(String texto) {
    mensajeLabel.setStyle("-fx-text-fill: green;");
    mensajeLabel.setText(texto);
}
```



Validación: al presionar Agregar con campos vacíos aparece el error en rojo. Con una fila seleccionada, Eliminar se habilita.

10. Paso 9: Guardar

Agregar y eliminar solo cambian la lista en memoria. El archivo se escribe **únicamente al presionar Guardar**:

```
@FXML
private void onGuardarClick() {
    try {
        repositorio.guardar(contactos);
        mostrarAviso("Guardado: " + contactos.size() + " contacto(s) en
datos/contactos.json.");
    } catch (IOException e) {
        mostrarError("No se pudo guardar el archivo: " + e.getMessage());
    }
}
```

> Si agregas contactos y cierras la ventana sin presionar Guardar, esos cambios se pierden. Es el comportamiento pedido; como desafío puedes avisar al usuario al cerrar.

11. Probar el resultado

1. Ejecuta la aplicación. La tabla parte vacía (aún no existe el JSON).
2. Presiona **Agregar** sin llenar nada: debe salir el error en rojo.
3. Agrega 2 o 3 contactos válidos. Cada uno aparece de inmediato en la tabla.

4. Presiona **Guardar**. Se crea `datos/contactos.json`.
5. Selecciona una fila y presiona **Eliminar**. Desaparece de la tabla (pero sigue en el archivo hasta que guardes de nuevo).
6. Cierra la aplicación y vuelve a abrirla: los contactos guardados aparecen cargados.

El archivo `datos/contactos.json` debe verse así:

```
[
  {
    "nombre": "Ana",
    "apellido": "Pérez",
    "correo": "ana.perez@correo.cl",
    "telefono": "911112222"
  },
  {
    "nombre": "Luis",
    "apellido": "Rojas",
    "correo": "luis.rojas@correo.cl",
    "telefono": "933334444"
  }
]
```

12. Código final del controller

`src/main/java/com/duoc/javafxtable/ContactoController.java`

```
package com.duoc.javafxtable;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;
import javafx.scene.control.cell.PropertyValueFactory;

import java.io.IOException;

public class ContactoController {

    @FXML
    private TextField nombreField;
    @FXML
    private TextField apellidoField;
    @FXML
    private TextField correoField;
    @FXML
    private TextField telefonoField;
    @FXML
    private Button eliminarButton;
    @FXML
    private Label mensajeLabel;

    @FXML
    private TableView<Contacto> tabla;
```

```
    @FXML
    private TableColumn<Contacto, String> nombreCol;
    @FXML
    private TableColumn<Contacto, String> apellidoCol;
    @FXML
    private TableColumn<Contacto, String> correoCol;
    @FXML
    private TableColumn<Contacto, String> telefonoCol;

    // Lista "observable": la tabla se redibuja sola cuando cambia
    private final ObservableList<Contacto> contactos = FXCollections.observableArrayList();
    private final ContactoRepositorio repositorio = new ContactoRepositorio();

    // Se ejecuta automáticamente después de que el FXML inyectó los campos
    @FXML
    private void initialize() {
        // El texto "nombre" se resuelve al getter getNombre() de Contacto
        nombreCol.setCellValueFactory(new PropertyValueFactory<>("nombre"));
        apellidoCol.setCellValueFactory(new PropertyValueFactory<>("apellido"));
        correoCol.setCellValueFactory(new PropertyValueFactory<>("correo"));
        telefonoCol.setCellValueFactory(new PropertyValueFactory<>("telefono"));

        contactos.addAll(repositorio.cargar());
        tabla.setItems(contactos);

        // El botón Eliminar queda deshabilitado mientras no haya una fila seleccionada

eliminarButton.disableProperty().bind(tabla.getSelectionModel().selectedItemProperty().isNu
ll());
    }

    @FXML
    private void onAgregarClick() {
        String nombre = nombreField.getText().trim();
        String apellido = apellidoField.getText().trim();
        String correo = correoField.getText().trim();
        String telefono = telefonoField.getText().trim();

        if (nombre.isEmpty() || apellido.isEmpty() || correo.isEmpty() ||
telefono.isEmpty()) {
            mostrarError("Debes completar todos los campos.");
            return;
        }
        if (!correo.contains("@") || !correo.contains(".")) {
            mostrarError("El correo no es válido.");
            return;
        }
        if (!telefono.matches("\\d{8,12}")) {
            mostrarError("El teléfono debe tener entre 8 y 12 dígitos.");
            return;
        }

        contactos.add(new Contacto(nombre, apellido, correo, telefono));
        limpiarFormulario();
        mostrarAviso("Contacto agregado. Presiona Guardar para escribirlo en el archivo.");
    }

    @FXML
    private void onEliminarClick() {
        Contacto seleccionado = tabla.getSelectionModel().getSelectedItem();
        if (seleccionado != null) {
            contactos.remove(seleccionado);
            mostrarAviso("Contacto eliminado. Presiona Guardar para escribirlo en el
```

```
archivo.");
        }
    }

    @FXML
    private void onGuardarClick() {
        try {
            repositorio.guardar(contactos);
            mostrarAviso("Guardado: " + contactos.size() + " contacto(s) en
datos/contactos.json.");
        } catch (IOException e) {
            mostrarError("No se pudo guardar el archivo: " + e.getMessage());
        }
    }

    private void limpiarFormulario() {
        nombreField.clear();
        apellidoField.clear();
        correoField.clear();
        telefonoField.clear();
        nombreField.requestFocus();
    }

    private void mostrarError(String texto) {
        mensajeLabel.setStyle("-fx-text-fill: red;");
        mensajeLabel.setText(texto);
    }

    private void mostrarAviso(String texto) {
        mensajeLabel.setStyle("-fx-text-fill: green;");
        mensajeLabel.setText(texto);
    }
}
```

13. Errores frecuentes

Síntoma	Causa probable
`package com.google.gson does not exist`	Falta la dependencia en el `pom.xml` o no recargaste Maven.
`module not found: com.google.gson`	Falta `requires com.google.gson;` en `module-info.java`.
`InaccessibleObjectException` al guardar o cargar	Falta `opens com.duoc.javafxtable to com.google.gson`.
La tabla se ve vacía pero hay datos	Falta `javafx.base` en el `opens`, o el texto de `PropertyValueFactory` no coincide con el getter.
`NullPointerException` en `initialize()`	Un `fx:id` del FXML no coincide con el nombre del campo del controller.
`LoadException ... onAgregarClick`	El método del `onAction` no existe o no tiene `@FXML`.
No encuentra `datos/contactos.json` al abrir	La ruta es relativa a donde ejecutas. Ejecuta desde la carpeta del proyecto (`mvn javafx:run` o, en IntelliJ, *Working directory* = raíz del proyecto).

14. Desafíos opcionales

- Pedir confirmación (`Alert`) antes de eliminar.
- Avisar al cerrar la ventana si hay cambios sin guardar.
- Impedir correos duplicados.
- Agregar un campo de búsqueda que filtre la tabla (`FilteredList`).
- Reemplazar el `TextField` del teléfono por un `ComboBox` de prefijo + número.